Teorie EGC

1. Volume de umbră

* formate dintr-o sursă de lumină și un poligon generator
* PUV = poligoane de umbră vizibile din poziția observatorului
  PUN = poligoane de umbră nevizibile din poziția observatorului
  → clasificarea lor se poate face pe baza normalelor la poligoane și a poziției observatorului (back face culling : front face și back face)
* fețele laterale ale volumului de umbră = poligoane de umbră
* poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă
* fie P un punct al unei suprafețe din scena 3D
  VP vectorul din poziția observatorului (V) în punctul P

P umbrit dacă numărul PUV intersectate de VP > numărul PUN intersectate de VP

→ poligoanelor PUV li se asociază +1
poligoanelor PUN li se asociază -1
} când VP traversează un poligon de umbră, contorului i se adună sau scade 1 în funcție de tipul poligonului

Contorul se inițializează cu numărul de volume de umbră care conțin poziția observatorului (>=0)

2. Algoritmul Z-Buffer

* algoritm de eliminare a FRAGMENTELOR NEVIZIBILE ALE PRIMITIVELOR GRAFICE
* executat de GPU, fiind integrat în PROCESUL DE RASTERIZARE A PRIMITIVELOR
* logică: dacă 2 fragmente aflate pe aceeași proiector, rezultate din rasterizarea a două primitive diferite, au coordonata Z diferită => fragmentul cu Z mai mic va fi afișat în pixelul (x,y)
* testul de vizibilitate (adâncime)
  - ține cont de poziția observatorului
  - este efectuat pe GPU
  - este efectuat în procesul de rasterizare
* Z-buffer-ul e actualizat pentru fiecare cadru imagine și memorează coordonatele Z ale fragmentelor vizibile într-un cadru imagine

3. Algoritmul BSP

* intrare: lista primitivelor ce compun scena, arborele BSP este independent de poziția observatorului
* fiecărui nod îi corespunde un plan de poziționare, care se împarte în 2 subspații - față / spate, fiecare generând un subarbore
* afișarea ține cont de poziția observatorului - front-to-back / back-to-front
* implementat într-un motor grafic
* structură de date ce poate fi folosită pentru gestionarea și optimizarea operațiilor ~~obiectelor~~ cu obiectele scenei 3D (alături de arborele BVH și arborele octal)
* permite eliminarea suprafețelor obturate și efectuarea operației de "Frustum culling" = eliminarea de obiecte / grupuri de obiecte care nu sunt incluse în volumul ~~vizual~~ vizual

* arborele BSP se obține partiționând recursiv spațiul 3D în semispații, folosind planele poligoanelor 3D
* la afișarea back-to-front poate fi dezactivat testul de adâncime
  la afișarea front-to-back nu sunt ~~suprascrise~~ suprascrise părțile poligoanelor aflate mai departe de observator
* eficient pentru afișarea scenelor ce conțin obiecte mari statice
* ~~trebe~~ ține cont de normalele la planele poligoanelor

~~* Algoritmul Pictorului~~

4. Algoritmul Pictorului
* algoritm din categoria "hidden surface removal"
* intrare: lista poligoanelor care alcătuiesc scena 3D
* idee: eliminarea părților nevizibile se rezolvă prin afișarea "din spate în față", la fel ca în pictură
* nu e necesar testul de adâncime, eficient când poligoanele sunt disjuncte pe z
* implementat într-un motor grafic
* în cazul unor scene dinamice, lista ordonată a poligoanelor trebuie modificată pentru fiecare cadru imagine

5. Introducerea umbrelor în algoritmul z-buffer - shadow mapping
* metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei
* în prima etapă → se construiește buffer-ul z al scenei văzută din poziția sursei de lumină, notat cu z_s (conține coordonatele fragmentelor (z) care primesc lumină de la sursă)
  în a doua etapă → se construiește imaginea văzută din poziția observatorului, utilizând bufferul z_s pentru determinarea punctelor aflate în umbră
* calitatea umbrelor depinde de rezoluția z-bufferului

6. Împărțirea perspectivă

* se aplică coordonatelor de decupare în urma proiecției
  ⇒ coordonate dispozitiv normalizate
* se trece de la coordonate omogene la coordonate carteziene
* vârfurile rezultate sunt asamblate în primitive, care sunt decupate la frontiera volumului vizual canonic
* transformarea de modelare (M) → vizualizare (V) → proiecție (P) → împărțirea perspectivă → transformare în poarta de afișare

7. Operații efectuate pe GPU

* eliminarea din banda grafică a fragmentelor nevizibile (Z-buffer) DA
* eliminarea din banda grafică a primitivelor care sunt complet obturate de alte primitive (occlusion culling) NU
* eliminarea din banda grafică a obiectelor care sunt complet în afara volumului vizual (Frustum culling) NU
* eliminarea din banda grafică a fețelor auto-obturate (Back face culling) DA

8. Tipuri de umbră

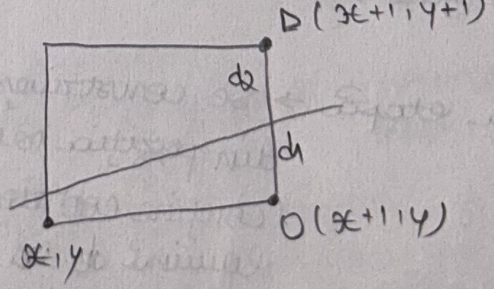
* umbră proprie: generată de obiectul însuși, care împiedică lumina să ajungă la unele din fețele sale: fețele umbrite de umbra proprie sunt fețele auto-obturate atunci când scena este văzută din poziția sursei de lumină
* umbra portată: umbra pe care un obiect o produce pe alte părți ale scenei, la care lumina nu ajunge din cauza obiectului

9. Bresenham original - pentru octantul 1

dacă $d_1 < d_2$ ⇒ următorul punct va fi $O(x+1, y)$

dacă $d_2 < d_1$ ⇒ următorul punct va fi $D(x+1, y+1)$

dacă $d_1 = d_2$ ⇒ oricare dintre O și D



10. Mapare inversă

* GL_NEAREST → considerăm culoarea texelului cu adresa cea mai apropiată
  → rapid, produce defecte de imagine
* GL_LINEAR → efectuăm o interpolare biliniară între culorile ~~cele mai~~ celor mai apropiați texeli
  → scade viteza de calcul a culorii, produce un efect de încețoșare

11. Modelul Lambert

* toate fragmentele unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală
* Legea lui Lambert:
  → reflexia difuză a luminii într-un punct al unei suprafețe
  - este dispersată regulat în toate direcțiile
  - este cu atât mai mare cu cât unghiul dintre normala la suprafață și vectorul de la punct la sursa de lumină este mai mic
  - are valoarea 0 dacă produsul scalar dintre normala la suprafață și vectorul de la punct la sursa de lumină este mai mic decât 0

12. Modelul Gouraud

* fiecărui vârf al poligonului îi este asociată o culoare → la momentul rasterizării
* culorile fragmentelor se obțin prin ~~interpolare~~ interpolarea liniară a culorilor din vârfuri
* culorile din vârfuri se calculează cu modelul de iluminare locală (în vertex shader), culoarea fiecărui fragment calculată în GPU de procesul de rasterizare
* pierdute calculul luminii reflectate speculor
* se pot aproxima corect în interiorul triunghiului: ~~ambientală~~ ~~difuză~~
  (componenta ambientală și emisivă
  → NU se pot aproxima corect: componenta speculară și factorul de atenuare pentru fragmentele din interiorul Δ

13. Modelul Phong

* fiecărui vârf îi este asociată o normală
* normala și poziția vârfului (coordonate globale) sunt ieșiri din vertex shader
* culorile fragmentului se calculează în fragment shader aplicând modelul de iluminare locală ↳ folosind normala fragmentului
* normala este normalizată în fragment shader
* permite redarea reflexiei speculare în orice fragment al unui poligon
* se poate calcula componenta speculară a reflexiei luminii într-un fragment al unei primitive
* pe baza performanței : Lambert < Gouraud < Phong

14. Modelul RGB - orientat către echipamente

* pe diagonala Alb - Negru - nuanțe de gri
* (1,0,0) Roșu — complementar cu (0,0,1) Cian

RGB
CMY

* spațiul culorilor reprezentat de un cub cu coordonate cuprinse între [0,0,0] și [1,1,1]
* Alb [1,1,1]
  Negru [0,0,0]

15. Modelul HSV - orientat către utilizator

* $0 \leq V \leq 1$, $0 \leq S \leq 1$, $0 \leq H \leq 360$
* culori complementare la 180° pe baza hexagonului
* saturația = dis(axa hexagonului, punct)
* alb : S = 0, V = 1
  gri : S = 0 , $0 \leq V \leq 1$
  culori pure : S = 1 , V = 1
  ↳ se află pe frontiera bazei hexagonului

16. Modelul HSL

* culorile primare cu saturație maximă și complementarele lor sunt reprezentate prin S = 1 , L = 0.5

17. Algoritmul Ray Tracing

* imaginea nu se obține prin rasterizarea primitivelor, ci prin intersecția scenei cu raze care pleacă din poziția observatorului
* depinde de direcția razei primare
* se pot reda umbre și reflexii ale luminii
* P - raza primară ce trece prin pixel și intersectează scena 3D
  T - raza transmisă calculată cu Snell → calculează contribuția luminii venite de la alte obiecte
  L - raza către sursa de lumină → din intersecția I a lui R cu un obiect din scenă
  R - raza reflectată speculor, simetrică față de N cu L

18. Eliminarea fețelor auto-obturate (BACK FACE CULLING) → pe GPU

* reduce substanțial numărul de poligoane rasterizate
* algoritm: observatorul și obiectul sunt raportați la același sistem de coordonate, conturul fiecărei fețe a obiectului este orientat în sens trigonometric atunci când poliedrul este văzut din exterior, observator situat în afara obiectului

19. Stencil Buffer (la volume de umbră)

* bufferul de marcaje (stencil buffer) - în GPU cu bufferul de imagine (frame buffer) și bufferul de adâncime (z-buffer), actualizate prin rasterizare

* DEPTH-PASS: stencil buffer-ul e modificat la reușita testului de adâncime
  → calculez dacă observatorul se află în mai multe volume de umbră e dificil
  → mai întâi desenăm fețele din față, apoi poligoanele din umbră, apoi fețele din spate

* DEPTH-FAIL: stencil buffer-ul e modificat când testul de adâncime eșuează
  → desenăm mai întâi fețele din spate, apoi poligoanele din umbră, apoi fețele din față

20. Efectul de spot

* în modelul de iluminare locală, sursa de lumină e considerată punctiformă; lumina produsă de o sursă punctiformă este omnidirecțională
* efectul de spot simulează o sursă de lumină care produce un con de lumină; numai suprafețele aflate în conul de lumină primesc lumină de la sursă
* conul de împrăștiere a luminii se calculează folosind:
  - → poziția spotului (sursei de lumină)
  - → direcția spotului
  - → împrăștierea luminii spotului (θspot sau cutoff)

21. Transparența

* majoritatea algoritmilor de determinare a vizibilității pot fi adaptați pentru a include transparența
* în OpenGL, transparența poate fi simulată prin amestecul dintre fragmentul curent și cea a pixelului care se afișează, folosind modelul (R, G, B, A) – A = opacitate { 0 transparent / 1 opac
* se face cu GL_BLEND() și glBlendFunc()
* operația de amestec: efectuată în procesul de rasterizare după testul de adâncime, înaintea actualizării valorii pixelului în buffer imagine

22. Ceața

* adaugă realism imaginii și ajută la eliminarea defectelor produse de decuparea la nivelul din spate al volumului vizual
* $0 \leq \phi \leq 1$ - factorul de ceață, proporțional cu distanța de la observator la punct ($\phi = 1$ opacitate)
* factorul de ceață liniar, acționează între 2 distanțe definite în sistemul de coordonate, măsurate pe direcția în care privește observatorul
* depinde de distanța dintre observator și punctul în care se calculează culoarea afectată de ceață.

23. Proiecții

Proiecții $R^3 \to R^2$ → proiecții perspectivă – centrul de proiecție este situat la distanță finită
– proiectorii sunt drepte convergente în centrul de proiecție

→ proiecții paralele – centrul de proiecție este la ~~infinit~~ infinit
– proiectorii sunt linii paralele și au direcția specifică (vectorul direcție de proiecție)

~~Proiecții perspectivă~~

Proiecții perspectivă

* efectul de micșorare: ~~dim~~ dimensiunea obiectului proiectat în 2D este invers proporțională cu distanța de la ~~ob~~ centrul de proiecție la obiect

* modifică unghiurile dintre dreptele care nu sunt paralele cu planul de proiecție
* rezultă imagini similare cu cele de la un aparat foto
* nu conservă paralelismul liniilor => nu este afină
* modifică lungimile laturilor în funcție de distanța lor față de centrul de proiecție

Proiecții paralele

* proiectorii sunt linii paralele de direcția dată : direcția de proiecție
* sunt afine, conservă paralelismul liniilor
* unghiurile se conservă doar pentru fețele obiectului paralele cu planul de proiecție

clasificare: · ORTOGONALE – proiectorii sunt ⊥ pe planul de proiecție
↳ ortografice : planul de proiecție ∥ cu un plan principal (⊥ pe o axă principală); conservă lungimea laturilor și a unghiurilor dintre laturi
↳ axonometrice : izometrice, dimetrice, trimetrice
planul de proiecție nu e ⊥ cu nicio axă
văd mai multe fețe (ca cele perspective) 9/10

· OBLICE - planul de proiecție ⊥ pe o axă principală
- direcția de proiecție nu e ⊥ pe planul de proiecție
→ CAVALIER (45°) - conservă lungimile laturilor ⊥ pe planul de proiecție
→ CABINET (63.43°) - jumătate de latură

în OpenGL : perspectivă și ortografică

24. ~~Fereastră~~ - poartă

*Scalare uniformă => $s = \min(sx, sy)$

$$sx = \frac{xpmax - xpmin}{xfmax - xfmin} \qquad sy = \frac{ypmax - ypmin}{yfmax - yfmin}$$

~~* factorul de~~

* factorii de translație : $tx = xpmin - s \cdot xfmin$
$ty = ypmin - s \cdot yfmin$

~~* aliniere~~

* afișare centrată în poartă => translație suplimentară față de OX și OY

$$tsx = \frac{xpmax - xpmin - s \cdot (xfmax - xfmin)}{2}$$

$$tsy = \frac{ypmax - ypmin - s \cdot (yfmax - yfmin)}{2}$$

* rezultat : $xp = xf \cdot s + tx + tsx$, $yp = yf \cdot s + ty + tsy$

~~xp = xf · s + tx + tsx~~
~~yp = yf · s + ty + tsy~~

* depinde de mărimea laturilor ferestrei și ale porții